

Assignment For Introduction to collection

Q1) Write Java code to define List. Insert 5 floating point numbers in List, and using an iterator, find the sum of the numbers in List.

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        List<Float> list = new ArrayList<>();

        list.add(10.5f);
        list.add(20.75f);
        list.add(5.25f);
        list.add(12.0f);
        list.add(8.5f);

        Iterator<Float> iterator = list.iterator();
        float sum = 0.0f;

        while(iterator.hasNext()){
            sum+=iterator.next();
        }

        System.out.println("Sum is :" + sum);
    }
}
```

h Main x

/home/aman/.jdk/openjdk-24.0.1/bin/java -javaagent:/snap/

Sum is :57.0

Q2) Given the following class Employee class{ Double Age; Double Salary; String Name} Design the class in such a way that the default sorting should work on firstname and lastname. Also, Write a program to sort Employee objects based on salary using Comparator.

```
class Employee implements Comparable<Employee> {
    Double age;
    Double salary;
    String firstName;
    String lastName;

    public Employee(String firstName, String lastName, Double age,
Double salary) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
        this.salary = salary;
    }

    public String toString() {
        return firstName + " " + lastName + " | Age: " + age + " |
Salary: " + salary;
    }

    @Override
    public int compareTo(Employee e) {
        int compare1 = this.firstName.compareTo(e.firstName);
        if (compare1 == 0) {
            return this.lastName.compareTo(e.lastName);
        } else {
            return compare1;
        }
    }
}
```

```
import java.sql.ClientInfoStatus;
import java.util.*;
class sortBySalary implements Comparator<Employee> {

    @Override
    public int compare(Employee o1, Employee o2) {
        return o1.salary.compareTo(o2.salary);
    }
}
```

```

    }
}

public class Main {
    public static void main(String[] args) {
        List<Employee> list = new ArrayList<>();

        list.add(new Employee("Akash", "Kumar", 23.0, 50000.0));
        list.add(new Employee("Aman", "Pal", 23.0, 55000.0));
        list.add(new Employee("Sahil", "Singh", 23.0, 60000.0));
        list.add(new Employee("Manish", "Yadav", 23.0, 40000.0));

        Collections.sort(list);
        System.out.println("Sorted by name \n");
        for(Employee emp: list){
            System.out.println(emp);
        }

        Collections.sort(list, new sortBySalary());
        System.out.println("Sorted By Salary");
        for(Employee emp: list) {
            System.out.println(emp);
        }
    }
}

```

Q3) Design a Data Structure SpecialStack that supports all the stack operations like push(), pop(), isEmpty(), isFull() and an additional operation getMin() which should return minimum element from the SpecialStack. (Expected complexity O(1))

```

import java.util.Stack;

class SpecialStack {
    Stack<Integer> mainStack = new Stack<>();
    Stack<Integer> minStack = new Stack<>();
    int maxSize;

    SpecialStack(int size) {
        this.maxSize = size;
    }
}

```

```
public void push(int x) {
    if (isFull()) {
        System.out.println("Stack is full. Cannot push " + x);
        return;
    }
    mainStack.push(x);

    if (minStack.isEmpty() || x <= minStack.peek()) {
        minStack.push(x);
    }
}

public int pop() {
    if (isEmpty()) {
        System.out.println("Stack is empty.");
        return -1;
    }

    int removed = mainStack.pop();
    if (removed == minStack.peek()) {
        minStack.pop();
    }

    return removed;
}

public int getMin() {
    if (minStack.isEmpty()) {
        System.out.println("Stack is empty.");
        return -1;
    }
    return minStack.peek();
}

public boolean isEmpty() {
    return mainStack.isEmpty();
}

public boolean isFull() {
    return mainStack.size() == maxSize;
}

public int peek() {
    if (isEmpty()) {
        System.out.println("Stack is empty.");
    }
}
```

```

        return -1;
    }
    return mainStack.peek();
}
}

```

Q4) Create class Employee with attributes name, age, designation and use instances of these class as keys in a Map and their salary as value

```

import java.util.*;
class Employee {
    String name;
    int age;
    String designation;

    public Employee(String name, int age, String designation) {
        this.name = name;
        this.age = age;
        this.designation = designation;
    }

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return false;
        Employee employee = (Employee) o;
        return age == employee.age && Objects.equals(name,
employee.name) && Objects.equals(designation, employee.designation);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, age, designation);
    }

    public String toString() {
        return name + " (" + age + ", " + designation + ")";
    }
}

```

```

import java.sql.ClientInfoStatus;

```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Map<Employee, Double> map = new HashMap<>();

        // Creating Employee objects
        Employee e1 = new Employee("Aman", 23, "Developer");
        Employee e2 = new Employee("Akash", 23, "Manager");
        Employee e3 = new Employee("Sahil", 23, "Tester");

        map.put(e1, 60000.0);
        map.put(e2, 80000.0);
        map.put(e3, 50000.0);

        for (Map.Entry<Employee, Double> entry : map.entrySet()) {
            System.out.println("Employee: " + entry.getKey() + " |
Salary: " + entry.getValue());
        }
    }
}
```